Lesson 09 · Data Access with EF Core

DbContext, LINQ-to-SQL, change tracking, concurrency, migrations and Dapper — EF Core 10 read through JPA/Hibernate, TypeORM and Flyway

Contents

1 · Why data access is where a .NET service shows its age 2
- 1.1 · The numbers to leave this lesson with 2

2 · The model — DbContext, entities and value objects 2
- 2.1 · Concept map — JPA, Hibernate, TypeORM and Flyway → EF Core 3
- 2.2 · The same mapping — JPA annotations vs an EF Core configuration class 4
- 2.3 · The context — a primary constructor, four DbSets, configurations by assembly scan 4
- 2.4 · Value objects — a record struct and a record, both complex types 4
- 2.6 · Six mapping tools — which one a codebase uses tells you its era 5

3 · Querying — LINQ becomes SQL, or it silently does not 6
- 3.2 · A projection and the SQL it becomes 7
- 3.3 · The IEnumerable trap — the same Count, two static types 7
- 3.4 · Commands to load N quotes with their lines 8
- 3.5 · Entities tracked after each 3.3 query 8
- 3.6 · The N+1 as a failing test, not a code-review opinion 8
- 3.7 · Five ways to load a quote's premium lines 8
- 3.8 · Raw SQL that stays parameterised 9

4 · Saving — unit of work, transactions and optimistic concurrency 9
- 4.1 · Entity states in the change tracker 9
- 4.2 · Accepting a quote — two statements, one SaveChanges, one transaction 10
- 4.3 · An application-managed concurrency token for a database without rowversion 10
- 4.4 · An underwriter and a broker edit the same quote 11
- 4.5 · No flush before a query — and identity resolution still applies 11
- 4.6 · One explicit transaction — ExecuteUpdate and SaveChanges commit or roll back together 12
- 4.7 · Persistence-context habits — what carries over from Hibernate, what to drop 12

5 · Migrations — dotnet ef instead of Flyway and Liquibase 12
- 5.1 · A generated migration — what you review in the pull request 13
- 5.2 · Four ways to apply migrations, next to the Flyway habit 13
- 5.3 · The CI gate, a SQL script that SQLite cannot make idempotent, and a bundle run twice 14

6 · Beyond tracked entities — Dapper, bulk statements, providers and DynamoDB 14
- 6.1 · A reporting query — Spring JdbcClient vs Dapper 15
- 6.2 · Expiring stale quotes — tracked SaveChanges vs one ExecuteUpdate 15
- 6.3 · Providers you will meet — and whether they support EF Core 10 15
- 6.4 · Commands to expire the stale quotes 16
- 6.5 · Which tool gives you what 16

7 · Testing data access — choose the fake that lies least 16
- 7.1 · The fixture — one open connection is one private database 17
- 7.2 · Counting commands with a DbCommandInterceptor 17
- 7.3 · The InMemory provider trap, pinned by a test 18
- 7.4 · SQLite stores decimal as TEXT — raw SQL sorts it as text 18
- 7.6 · What the application may still do — for each kind of test double 19

8 · Visual Basic alongside — the runtime is shared, the tooling is not 19
- 8.1 · One grouping query — C# method syntax vs VB query syntax 19
- 8.2 · What EF Core receives from VB — and the SQL it sends 20
- 8.3 · The EF Core tools refuse a VB project 20

9 · Hands-on — run the lesson-09 samples 20
- 9.1 · Commands 20
- 9.2 · What you should see 21

1 · Why data access is where a .NET service shows its age

You have shipped JPA entities, TypeORM repositories, Flyway migrations and hand-tuned reporting SQL, so an ORM is not new to you — what is new is where EF Core draws its lines. A quoting service for a motor-insurance platform with dozens of partner integrations lives or dies on its data layer: the premium lines, the quote status machine, the concurrent underwriter and broker edits. EF Core covers the same ground as Hibernate, but the persistence context is shorter-lived, lazy loading is off, queries never flush pending changes, and migrations are generated C# rather than hand-written SQL.

The fastest way to learn EF Core is to count the SQL it sends. Every sample in this lesson runs against SQLite in memory with an interceptor that records each command, so the N+1 problem, the cost of a tracked bulk update and the difference between `IQueryable` and `IEnumerable` are numbers you can assert in a test, not rules of thumb.

EF Core is not the only tool, and choosing between the tools is the architecture work. Dapper is the `JdbcTemplate` of .NET; `ExecuteUpdate` is the `@Modifying @Query`; DynamoDB has no EF Core provider, so the AWS SDK's own persistence model takes EF's place there. Section 6 lays the options side by side.

Legacy estates add a VB dimension. Visual Basic code can query a C# `DbContext` directly, but the EF Core design-time tools generate only C#. The practical architecture — model and migrations in a C# library, VB consumers — is shown in section 8.

1.1 · The numbers to leave this lesson with

EF CORE 10 · LTS	LOAD 20 QUOTES + LINES	EXPIRE 9 QUOTES	TEST CASES	INMEMORY PROVIDER
to Nov 2028	**21 → 1**	**10 → 1**	**25**	**discouraged**
ends 10 Nov 2028 · 781 days from build · verified	commands: N+1 vs Include · measured	commands: SaveChanges vs ExecuteUpdate · measured	L09.Data.Tests on SQLite in-memory · measured	for testing, per Microsoft · verified

KEY: • Same idea • Renamed • Different • Trap • Verified • Estimate • Measured C# architecture legacy

i Before you start

1. **You need lesson 04 (Collections, LINQ & Functional C#) for LINQ and expression trees, and lesson 08 (ASP.NET Core Web APIs) for dependency injection** — `AddDbContext` registers a context per request scope, which is the lifetime this lesson assumes.
2. **You will build five projects:** `L09.Data` (C# library: `MotorQuoteDbContext`, entities, configuration, a committed migration, Dapper report), `L09.Data.Tests` (25 xUnit cases on SQLite in-memory), `L09.QueryConsole` (prints the SQL), `L09.VbReport` (VB console querying the C# context) and `L09.VbModel` (a VB-declared context the tools refuse).
3. **Packages:** EF Core 10.0.12 (SQLite provider), Dapper 2.1.86, and the `dotnet-ef` 10.0.12 local tool pinned in `samples/dotnet-tools.json` — run `dotnet tool restore` once in the samples folder. No database server, Docker or cloud account is needed. EF Core 10 requires .NET 10 and is supported until 10 November 2028 (EF Core releases and planning); the .NET support policy ends .NET 10 itself four days later, on 14 November 2028 (lesson 01 (The .NET Platform Map)) — two published tables, not two support windows.
4. The seed data is priced with the **one illustrative tariff every lesson in this track uses** — not a real tariff. Three or more claims in five years is a decline stored as data: `QuoteStatus.Declined`, a zero total and one line, never an exception; a test prices the track's worked example at 8,933.71 THB so this lesson cannot drift from the others. Facts that change with releases are • Verified and linked; numbers from runs of the samples are • Measured; judgements are • Estimate. Kotlin, Java and TypeScript panels are for comparison and are not compiled.

2 · The model — DbContext, entities and value objects

A `DbContext` is a Hibernate `Session` you are expected to throw away after one unit of work. It holds the identity map and change tracker, exposes `DbSet<T>` roots for queries, and writes everything in `SaveChanges`. It is not thread-safe, so a web app gets one per request from `AddDbContext` — the DI scope rules are lesson 08 (ASP.NET Core Web APIs) — and a second operation started before the first completes throws (DbContext lifetime). Await every async call immediately (lesson 05 (Async, Tasks & Concurrency)). For parallel work or a `BackgroundService`, inject `IDbContextFactory<T>` and create one context per task. `AddDbContextPool` replaces `AddDbContext` where allocation matters: it resets a disposed context and hands the same instance out again instead of building a new one. That instance outlives the request, so `OnConfiguring` runs once and per-request state — a tenant id — needs a scoped factory over the pool (DbContext pooling).

The mapping lives beside the entities, not on them. JPA taught you to annotate the entity. EF Core can read attributes too, but the idiomatic shape is a plain class plus an `IEntityTypeConfiguration<T>` written with the Fluent API, which wins when both are present. The domain types stay persistence-ignorant, and what attributes cannot express — value conversions, query filters, seed data — sits in one place.

Value objects are complex types, and EF Core 10 finally lets `Money` be a struct. Complex types (EF Core 8) are the `@Embeddable` of EF: no key, value semantics, columns on the owner's table. EF Core 10 added struct complex types, optional complex types and JSON mapping, and Microsoft advises users of the older *owned entity types* to switch (What's new in EF Core 10). The samples map `Money` as a `readonly record struct`, stored as `Total_Amount` and `Total_Currency`.

2.1 · Concept map — JPA, Hibernate, TypeORM and Flyway → EF Core

You already know	In .NET	Kind	Watch for
`EntityManager` / Hibernate `Session`	`DbContext`	• Renamed	Short-lived unit of work; not thread-safe; scoped per request — lesson 08 (ASP.NET Core Web APIs)
`@Entity` + field annotations · TypeORM `@Entity/@Column`	Plain class + `IEntityTypeConfiguration<T>`	• Different	Fluent API overrides attributes; no marker needed on the class
`@Embeddable` / `@Embedded`	Complex type — `ComplexProperty`	• Same idea	EF Core 10: structs and optional values; columns named `Total_Amount`
`@Enumerated(EnumType.STRING)`	`HasConversion<string>()`	• Renamed	The column sorts as text, not in declaration order
`@Version` (Hibernate increments it)	`IsConcurrencyToken()` / `[Timestamp]`	• Trap	EF never increments a token for you; only a database-generated one changes by itself — SQL Server `rowversion`, PostgreSQL `xmin` through Npgsql
Lazy `@OneToMany` through proxies	No lazy loading unless enabled — Proxies package or `ILazyLoader`	• Trap	A forgotten `Include` gives an empty list — no exception (Lazy loading)
`JOIN FETCH` / `@EntityGraph`	`Include` / `ThenInclude`	• Renamed	Add `AsSplitQuery` when several collections multiply rows
JPQL · Criteria API · TypeORM `QueryBuilder`	LINQ over `IQueryable<T>`	• Different	The same lambda typed as `IEnumerable` filters in memory
Flush before a query (`FlushMode.AUTO`)	No automatic flush	• Trap	Queries read the database; tracked instances keep their unsaved values (4.5)
`@Transactional` (declarative)	No declarative transactions — `SaveChanges` is one; else `BeginTransactionAsync()`	• Different	Write the transaction in code, commit explicitly, dispose to roll back (4.6)
`@Modifying @Query("update …")`	`ExecuteUpdate` / `ExecuteDelete`	• Same idea	Skips the change tracker and concurrency tokens; Dapper is the `JdbcClient` (6.1)
TypeORM `migration:generate` · Liquibase `diff-changelog`	`dotnet ef migrations add`	• Same idea	Both write the migration from a model diff; EF diffs against a committed snapshot, TypeORM against the live schema
Flyway `V1__init.sql` · Liquibase changelog	Migrations: generated C# + model snapshot	• Different	Hand-written SQL becomes generated C#; SQL is derived, not authored
H2 in-memory database for tests	SQLite in-memory	• Same idea	The EF Core InMemory provider is not a database — avoid it

2.2 · The same mapping — JPA annotations vs an EF Core configuration class

Kotlin — for comparison JPA annotations on the entity

```kotlin
@Entity
@Table(name = "quotes")
class Quote(
    @Id @GeneratedValue
    var id: Long? = null,
    @Column(unique = true, length = 12)
    var reference: String,
    @Enumerated(EnumType.STRING)
    @Column(length = 12)
    var coverage: CoverageClass,
    @Embedded
    @AttributeOverride(name = "amount",
        column = Column(precision = 12, scale = 2))
    var total: Money,
    @Embedded
    var driver: Driver,
    @ManyToOne(optional = false)
    var vehicle: Vehicle,
    @OneToMany(cascade = [CascadeType.ALL])
    @JoinColumn(name = "quote_id")
    val lines: MutableList<PremiumLine> = mutableListOf(),
    @Version
    var version: Long = 0,
)
```

C# samples/L09.Data/Configuration.cs · #region quote-config

```csharp
internal sealed class QuoteConfiguration
    : IEntityTypeConfiguration<Quote>
{
    public void Configure(EntityTypeBuilder<Quote> b)
    {
        b.HasIndex(q => q.Reference).IsUnique();
        b.Property(q => q.Reference).HasMaxLength(12);
        b.Property(q => q.Coverage)
            .HasConversion<string>().HasMaxLength(12);
        b.Property(q => q.Status)
            .HasConversion<string>().HasMaxLength(10);
        b.ComplexProperty(q => q.Total, m =>
        {
            m.Property(x => x.Amount).HasPrecision(12, 2);
            m.Property(x => x.Currency).HasMaxLength(3);
        });
        b.ComplexProperty(q => q.Driver);
        b.HasOne(q => q.Vehicle).WithMany()
            .OnDelete(DeleteBehavior.Restrict);
        b.HasMany(q => q.Lines).WithOne()
            .HasForeignKey(l => l.QuoteId);
        b.Property(q => q.Version).IsConcurrencyToken();
    }
}
```

Line for line the same decisions, in a different place. `HasIndex().IsUnique()` is `unique = true`; `HasConversion<string>()` is `EnumType.STRING`; `ComplexProperty` is `@Embedded`. Two lines differ in behaviour: `OnDelete(Restrict)` is explicit because EF's default for a required relationship is cascade, and `IsConcurrencyToken()` only adds the `WHERE Version = @original` check — unlike `@Version`, nothing increments it (4.3 does).

2.3 · The context — a primary constructor, four DbSets, configurations by assembly scan

C# samples/L09.Data/MotorQuoteDbContext.cs · #region context

```csharp
// The EntityManager of .NET: a short-lived unit of work that owns the
// change tracker. Create one per request (DI scope) — never share it.
public sealed partial class MotorQuoteDbContext(
    DbContextOptions<MotorQuoteDbContext> options) : DbContext(options)
{
    public DbSet<Vehicle> Vehicles => Set<Vehicle>();
    public DbSet<Quote> Quotes => Set<Quote>();
    public DbSet<PremiumLine> PremiumLines => Set<PremiumLine>();
    public DbSet<Policy> Policies => Set<Policy>();

    protected override void OnModelCreating(ModelBuilder model) =>
        model.ApplyConfigurationsFromAssembly(GetType().Assembly);
}
```

No connection string and no provider in the class. The options arrive through the constructor, so the same context runs on SQLite in tests and on another provider in production. `partial` lets 4.3 add the token-stamping override in a second file.

2.4 · Value objects — a record struct and a record, both complex types

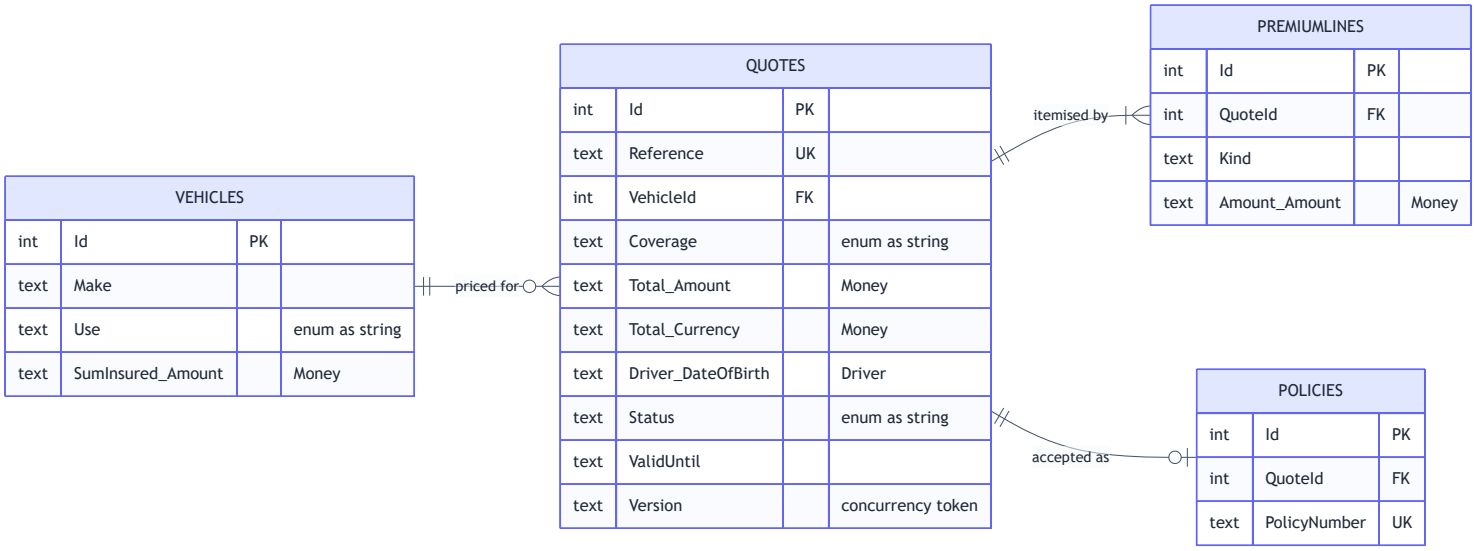
C# samples/L09.Data/Domain.cs · #region money

```csharp
// A value object: no key, compared by value. Configuration.cs maps it
// with ComplexProperty (complex types are never found by convention):
// columns on the owning row, no table.
public readonly record struct Money(decimal Amount, string Currency)
{
    public static Money Thb(decimal amount) => new(amount, "THB");
}

// A record class works too (Driver_* columns): also ComplexProperty.
public sealed record Driver(
    DateOnly DateOfBirth, int LicenceYears, int ClaimsLast5Years);
```

Value semantics are the point. Assigning one quote's `Total` to another copies the values; with an owned entity type the same assignment fails because the instance has an identity (Complex types). `decimal`, never `double`, for money — lesson 02 (C# Language Essentials).

2.5 · The schema the committed migration creates



tables from the InitialCreate migration · complex-type columns carry their owner's prefix (Total_, Driver_, SumInsured_, Amount_) · decimals are TEXT because this is SQLite · selected columns shown

2.6 · Six mapping tools — which one a codebase uses tells you its era

model view

1 Conventions C# • Same idea

EF Core builds most of the model without being told: `Id` is the key, a `List<PremiumLine>` navigation is a one-to-many, public properties become columns.

JPA The defaults behind `@Entity` — but EF needs no marker on the class

FINDS Every type reachable from a `DbSet` or from another entity's navigation

WATCH Enums map to integers; `Money` is not discovered as a complex type — configure it

SAMPLE `Vehicle.Id`, `Quote.VehicleId` and the table names

2 Data annotations C# • Renamed

Attributes on the class — `[Key]`, `[MaxLength]`, `[ConcurrencyCheck]`, `[Timestamp]`, `[ComplexType]`.

JPA `@Id`, `@Column(length=…)`, `@Version`, `@Embeddable`

COSTS None for these five — they are BCL types. EF's own `[Index]`, `[Owned]`, `[Precision]` live in `Microsoft.EntityFrameworkCore.Abstractions` and do put EF in the domain assembly

RULE Fluent API configuration overrides an attribute when both are present (Creating and configuring a model)

3 Fluent API in IEntityTypeConfiguration C# architecture • Different

One configuration class per entity, applied by `ApplyConfigurationsFromAssembly`.

JPA `orm.xml`, rarely used — annotations won; in EF Core the Fluent API is the idiomatic default

GIVES Persistence-ignorant entities, and what annotations cannot say (value conversions, query filters, seed data)

SAMPLE `QuoteConfiguration` in 2.2

2.6 · Six mapping tools (continued)

model view

4 Complex types C# • Same idea

A value object with no key, stored as columns of its owner's table (or one JSON column).

JPA	`@Embeddable` / `@Embedded`
SINCE	EF Core 8; structs, optional values and JSON mapping in EF Core 10 (Complex types)
LIMITS	No navigations inside; collections of complex types only as JSON
SAMPLE	`Money` → `Total_Amount`, `Total_Currency`

5 Owned entity types C# legacy • Trap

The older way to embed a value object — everywhere in codebases written before EF Core 8.

LOOKS	Like `@Embeddable`: `OwnsOne(q => q.Total)`
IS	An entity with a hidden key and reference semantics — one instance cannot be assigned to two properties
ADVICE	Microsoft now recommends complex types for value objects (What's new in EF Core 10)

6 Value converters C# • Renamed

Store a property as another type: `HasConversion<string>()` on an enum.

JPA	`@Enumerated(STRING)` · `AttributeConverter`
WATCH	Strings sort as text: `ORDER BY Coverage` puts `Class3` before `Class3Plus` (6.1 output)
SAMPLE	`Coverage`, `Status` and `Use`

3 · Querying — LINQ becomes SQL, or it silently does not

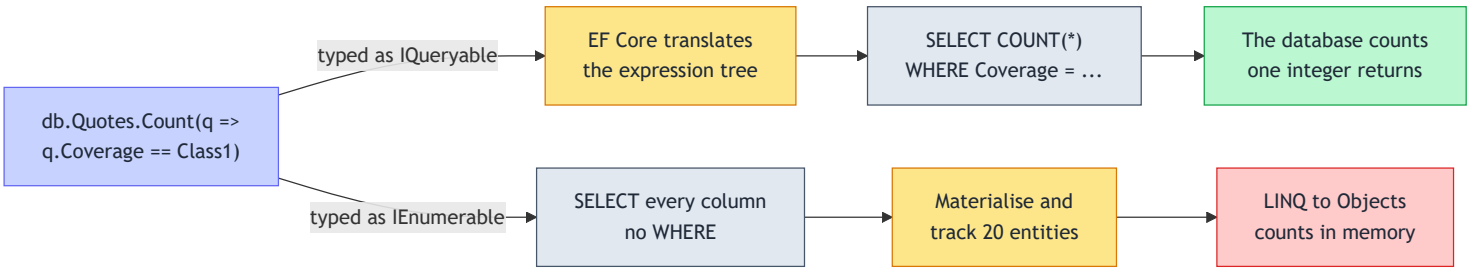
A LINQ query over a `DbSet` is data, not code: the compiler builds an expression tree, and EF Core translates it to SQL when you enumerate. This is lesson 04 (Collections, LINQ & Functional C#) put to work. `ToQueryString()` shows the SQL without running it — use it the way you used Hibernate's `show_sql`, but per query and in a test.

The static type decides where the filter runs. Assign `db.Quotes` to an `IEnumerable<Quote>` — a repository return type, a helper parameter — and the next `Where` or `Count` binds to LINQ to Objects: EF sends `SELECT` of every column of every row, tracks each entity, and filters in memory. It compiles, it passes a small test, and it is the most common EF performance bug.

Related data loads only when you ask for it. There is no lazy loading unless you turn it on — the Proxies package with `virtual` navigations, or `ILazyLoader` injected into the entity (Lazy loading) — so without `Include`, `quote.Lines` is simply empty. Loading lines quote-by-quote is the N+1; `Include` is one JOIN; `AsSplitQuery` trades one query for one per collection to avoid the cartesian explosion of several collection JOINs (Single vs. split queries).

Reads should be projections or no-tracking. A `Select` into a record reads only the named columns and is never tracked; `AsNoTracking()` keeps entities but skips the snapshot. Tracked queries are for data you are about to change.

3.1 · Two paths for one lambda



indigo = your query · amber = EF Core work · slate = the SQL sent · green = the database does the work · red = the work happens in your process instead

3.2 · A projection and the SQL it becomes

C# — samples/L09.Data/QuoteQueries.cs · #region projection

```
// Reads only the named columns; never tracked.
// q.Lines.Count becomes a COUNT(*) subquery.
public static IQueryable<QuoteSummary> Summaries(
    this MotorQuoteDbContext db, CoverageClass coverage) =>
    db.Quotes
        .Where(q => q.Coverage == coverage)
        .OrderBy(q => q.Reference)
        .Select(q => new QuoteSummary(
            q.Reference,
            q.Vehicle.Make,
            q.Total.Amount,
            q.Lines.Count));
```

SQL — ToQueryString() — captured from L09.QueryConsole

```
.param set @coverage 'Class1'
SELECT "q"."Reference", "v"."Make", "q"."Total_Amount", (
    SELECT COUNT(*)
    FROM "PremiumLines" AS "p"
    WHERE "q"."Id" = "p"."QuoteId")
FROM "Quotes" AS "q"
INNER JOIN "Vehicles" AS "v" ON "q"."VehicleId" = "v"."Id"
WHERE "q"."Coverage" = @coverage
ORDER BY "q"."Reference"
```

Three things to read in the SQL. The navigation `q.Vehicle.Make` became an `INNER JOIN` — no `Include` needed inside a projection. `q.Lines.Count` became a correlated `COUNT(*)` subquery instead of loading lines. The enum argument is a parameter, `@coverage`, so one cached plan serves every coverage class. Nothing selected here is tracked.

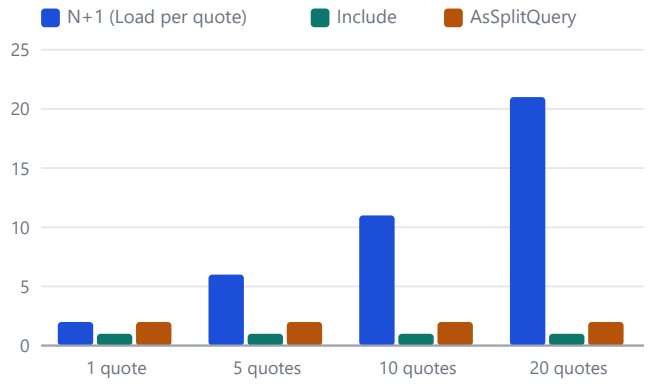
3.3 · The IEnumerable trap — the same Count, two static types

C# — samples/L09.QueryConsole/Program.cs · #region enumerable-vs-queryable

```
await using (var db = sqlite.NewContext())
{
    IEnumerable<Quote> all = db.Quotes;   // LINQ to Objects
    var n = all.Count(q => q.Coverage == CoverageClass.Class1);
    Report("IEnumerable", n, db);
}
await using (var db = sqlite.NewContext())
{
    IQueryable<Quote> query = db.Quotes;  // LINQ to SQL
    var n = query.Count(q => q.Coverage == CoverageClass.Class1);
    Report("IQueryable", n, db);
}
await using (var db = sqlite.NewContext())
{
    var rows = await db.Quotes.AsNoTracking()   // entities, no snapshot
        .Where(q => q.Coverage == CoverageClass.Class1)
        .ToListAsync();
    Report("AsNoTracking", rows.Count, db, printSql: false);
}
```

Same answer, very different work. Both print `count 5`. The `IEnumerable` version sent a `SELECT` of all twelve columns with no `WHERE` and left **20** entities tracked; the `IQueryable` version sent `SELECT COUNT(*) … WHERE "q"."Coverage" = 'Class1'` and tracked **0**. An `AsNoTracking()` list of the same 5 quotes also tracked 0 • **Measured**. Repository methods that return `IEnumerable` are fine only once the query has already been materialised.

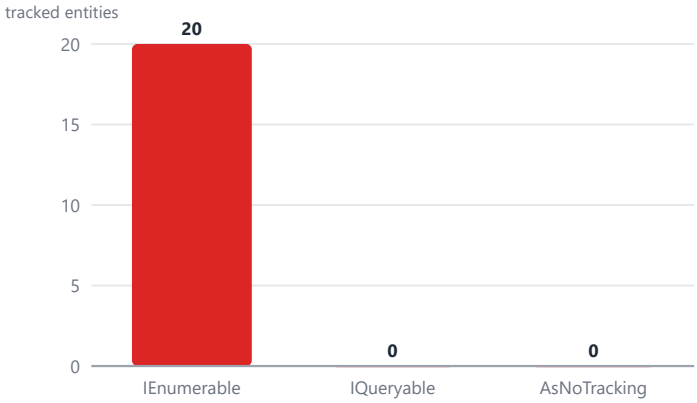
3.4 · Commands to load N quotes with their lines



SQL commands recorded by the SqlCommandLog interceptor · SQLite in-memory · measured

N+1 grows with the data; the other two do not. Loading 20 quotes line-by-line cost 21 commands; `Include` stayed at 1 and `AsSplitQuery` at 2 (one per collection level). On a cloud database every extra command is a network round trip. • Measured

3.5 · Entities tracked after each 3.3 query



ChangeTracker.Entries() after each query in 3.3 · 20 seeded quotes · measured

Tracking is memory and CPU you pay for on every read. The `IEnumerable` count materialised and snapshotted the whole table to return one integer. • Measured

3.6 · The N+1 as a failing test, not a code-review opinion

C# samples/L09.Data.Tests/QueryShapeTests.cs · #region command-counts

```
[Theory]
[InlineData(1)]
[InlineData(5)]
[InlineData(20)]
public async Task Loading_lines_per_quote_costs_one_command_each(int n)
{
    using var sqlite = new SqliteDb().Seed(n);
    await using var db = sqlite.NewContext();

    foreach (var quote in await db.Quotes.ToListAsync())
        await db.Entry(quote).Collection(q => q.Lines).LoadAsync();

    Assert.Equal(n + 1, sqlite.Log.Count);          // the N+1
}

[Theory]
[InlineData(1)]
[InlineData(20)]
public async Task Include_loads_the_same_graph_in_one_command(int n)
{
    using var sqlite = new SqliteDb().Seed(n);
    await using var db = sqlite.NewContext();

    var quotes = await db.Quotes.Include(q => q.Lines).ToListAsync();

    Assert.Equal(1, sqlite.Log.Count);
    Assert.All(quotes, q => Assert.NotEmpty(q.Lines));
}
```

Assert on the command count. The theory proves the cost grows as `n + 1`; the second test proves `Include` loads the same graph in one command. A later change that drops the `Include` fails a test instead of a dashboard. The interceptor that counts is shown in 7.2.

3.7 · Five ways to load a quote's premium lines

Approach	JPA / Hibernate habit	Commands for 20 quotes	Use it when
Nothing — no `Include`	Lazy collection proxy	1 · `Lines` stays empty • trap	Never, if you read `Lines` — EF raises no error
`Entry(q).Collection(…).LoadAsync()` in a loop	Touching a lazy collection per row	21 • measured	One quote at a time, never in a loop

Approach	JPA / Hibernate habit	Commands for 20 quotes	Use it when
`Include(q => q.Lines)`	`JOIN FETCH` · `@EntityGraph`	1 • measured	One collection level; the default choice
`Include(…).AsSplitQuery()`	Hibernate `@Fetch(SUBSELECT)`	2 • measured	Several sibling collections; add a unique `OrderBy` with paging on EF Core 9 or older
`Select` into a record	DTO projection with `select new`	1 · counts as subqueries	Read-only screens, APIs and reports — no tracking at all

3.8 · Raw SQL that stays parameterised

C# samples/L09.Data/QuoteQueries.cs · #region from-sql

```
// FromSql takes a FormattableString: {make} is sent as a parameter,
// never spliced into the SQL text.
public static IQueryable<Vehicle> VehiclesByMake(
    this MotorQuoteDbContext db, string make) =>
    db.Vehicles.FromSql($"SELECT * FROM Vehicles WHERE Make = {make}");
```

The $ is not string interpolation here. `FromSql` takes a `FormattableString` and sends `{make}` as a `DbParameter` (SQL queries). Fed `Toyota' OR '1'='1`, the console captured `SELECT * FROM Vehicles WHERE Make = @p0` and returned 0 rows. `FromSqlRaw` with concatenation is the injectable variant; EF Core 10 ships an analyzer warning for it (What's new in EF Core 10).

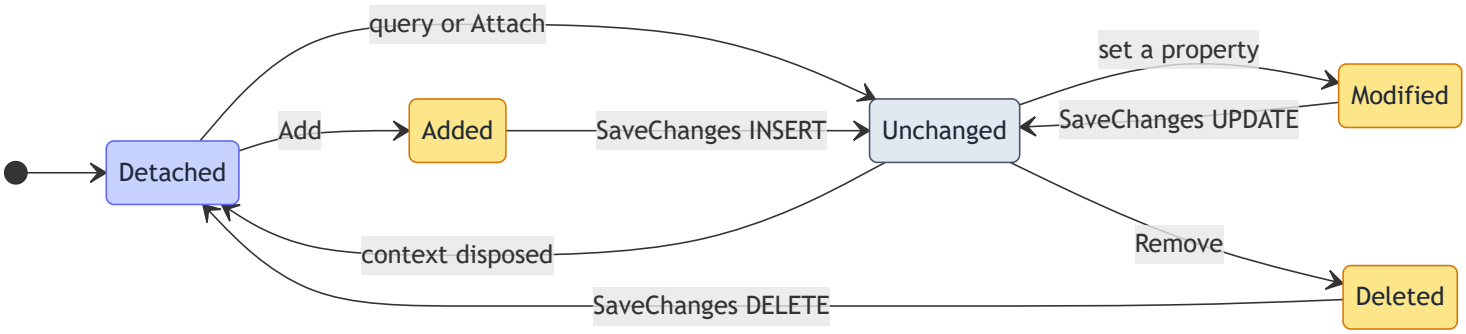
4 · Saving — unit of work, transactions and optimistic concurrency

SaveChanges is flush and commit in one call. The change tracker compares each tracked entity with the snapshot taken when it was loaded, then sends every INSERT, UPDATE and DELETE inside one transaction. Accepting a quote — update the quote, insert the policy — is one `SaveChangesAsync`, and either both rows change or neither does. There is no `@Transactional`: several `SaveChanges` or `ExecuteUpdate` calls that must succeed together need an explicit `BeginTransactionAsync` (4.6).

EF Core never flushes for you. Hibernate flushes pending changes before a query so the query sees them; EF Core sends the query as-is. A tracked quote you changed but did not save is invisible to a `WHERE` clause — yet querying its row again hands back the same tracked instance with the unsaved value. Test 4.5 pins both behaviours down.

Optimistic concurrency is a `WHERE` clause and a row count. Mark a property as a concurrency token and EF adds `AND Version = @original` to UPDATE and DELETE; zero rows affected becomes `DbUpdateConcurrencyException` — Hibernate's `OptimisticLockException`. The token has to change on every write: SQL Server's `rowversion` and PostgreSQL's `xmin` (mapped by Npgsql, Npgsql concurrency tokens) do that in the database, but SQLite has no database-generated token (Handling concurrency conflicts), so the sample stamps a new `Guid` inside `SaveChanges`.

4.1 · Entity states in the change tracker



indigo = not tracked · slate = in sync with the database · amber = a pending write that SaveChanges will send

4.2 · Accepting a quote — two statements, one SaveChanges, one transaction

C# — samples/L09.Data/QuoteWorkflow.cs · #region unit-of-work

```
public static async Task<Policy> AcceptAsync(
    MotorQuoteDbContext db, string reference, DateOnly today)
{
    var quote = await db.Quotes.SingleAsync(q => q.Reference == reference);
    if (quote.Status != QuoteStatus.Quoted)
        throw new InvalidOperationException($"{reference} is {quote.Status}");

    quote.Status = QuoteStatus.Accepted;       // tracked change -> UPDATE
    var policy = new Policy
    {
        PolicyNumber = "P-" + reference[2..],
        QuoteId = quote.Id,
        Inception = today,
        Expiry = today.AddYears(1),
    };
    db.Policies.Add(policy);                    // new entity -> INSERT

    await db.SaveChangesAsync();   // both statements, one transaction
    return policy;
}
```

No repository `save(quote)` call. Changing `quote.Status` on a tracked entity is enough for an UPDATE; `Policies.Add` queues the INSERT. The test `Accepting_a_quote_writes_the_update_and_the_insert_together` reads both back through a fresh context.

4.3 · An application-managed concurrency token for a database without rowversion

C# — samples/L09.Data/MotorQuoteDbContext.Versioning.cs · #region stamp-version

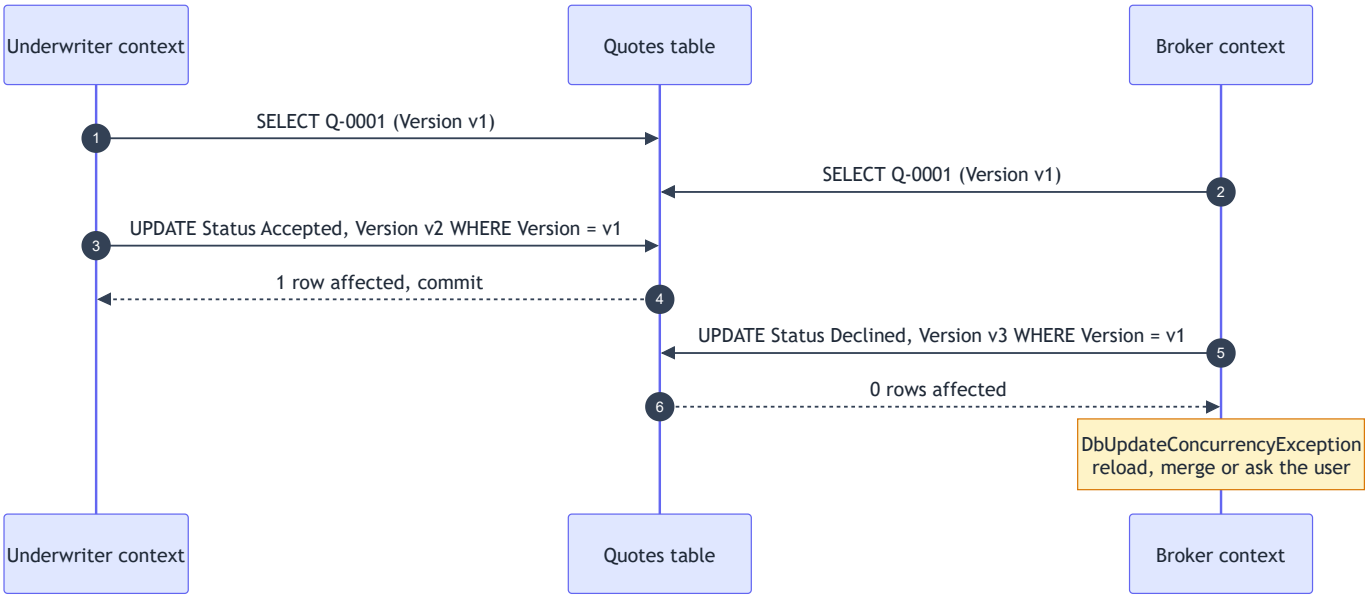
```
// SQLite has no rowversion, so the token is application-managed:
// every quote that SaveChanges is about to write gets a new Guid.
// The UPDATE still filters on the ORIGINAL value that was read.
public sealed partial class MotorQuoteDbContext
{
    public override Task<int> SaveChangesAsync(
        bool acceptAllChangesOnSuccess,
        CancellationToken cancellationToken = default)
    {
        StampVersions();
        return base.SaveChangesAsync(acceptAllChangesOnSuccess, cancellationToken);
    }

    private void StampVersions()
    {
        foreach (var entry in ChangeTracker.Entries<Quote>())
        {
            if (entry.State is EntityState.Added or EntityState.Modified)
                entry.Entity.Version = Guid.NewGuid();
        }
    }

    // ...and the synchronous SaveChanges(bool) override calls StampVersions() too.
```

The original value is still what the UPDATE filters on. EF captured `Version` when the quote was read; replacing it just before saving writes the new value while the `WHERE` clause keeps the old one. On SQL Server you would map a `byte[]` with `IsRowVersion()` instead (on PostgreSQL a `uint` for `xmin`) and delete this file. `ExecuteUpdate` bypasses this override — which is why 6.2 rotates the token itself.

4.4 · An underwriter and a broker edit the same quote



two DbContext instances, one row · the second UPDATE matches zero rows because its WHERE still carries the version it read · asserted by the test The_second_writer_gets_a_concurrency_exception

4.5 · No flush before a query — and identity resolution still applies

C# samples/L09.Data.Tests/SavingTests.cs · #region no-auto-flush

```
[Fact]
public async Task A_query_reads_the_database_not_unsaved_changes()
{
    using var sqlite = new SqliteDb().Seed(3);
    await using var db = sqlite.NewContext();
    var quote = await db.Quotes.SingleAsync(q => q.Reference == "Q-0001");

    quote.Status = QuoteStatus.Declined;       // tracked, not saved
    var declined = await db.Quotes
        .CountAsync(q => q.Status == QuoteStatus.Declined);
    var again = await db.Quotes.SingleAsync(q => q.Id == quote.Id);

    Assert.Equal(0, declined);                 // no flush before a query
    Assert.Same(quote, again);                 // identity resolution
    Assert.Equal(QuoteStatus.Declined, again.Status);
}
```

> **Two Hibernate instincts, tested.** The count reads the database, where Q-0001 is still `Quoted`, so it returns 0. Re-reading the row by id returns the *same* tracked object with the unsaved `Declined` — the identity map you know from the persistence context. Save before you query when the query must see your change.

4.6 · One explicit transaction — ExecuteUpdate and SaveChanges commit or roll back together

C# samples/L09.Data/QuoteWorkflow.cs · #region explicit-transaction

```
// Accepting one quote declines the other open quotes on the same vehicle.
// ExecuteUpdate and SaveChanges are separate commands, so one explicit
// transaction makes them succeed or fail together.
public static async Task AcceptAndDeclineRivalsAsync(
    MotorQuoteDbContext db, string reference, DateOnly today)
{
    await using var tx = await db.Database.BeginTransactionAsync();

    var quote = await db.Quotes.SingleAsync(q => q.Reference == reference);
    await db.Quotes
        .Where(q => q.VehicleId == quote.VehicleId && q.Id != quote.Id
                 && q.Status == QuoteStatus.Quoted)
        .ExecuteUpdateAsync(s => s
            .SetProperty(q => q.Status, QuoteStatus.Declined)
            .SetProperty(q => q.Version, Guid.NewGuid()));

    await AcceptAsync(db, reference, today);   // its SaveChanges joins tx
    await tx.CommitAsync();                    // no commit = rollback on dispose
}
```

This is the `@Transactional` you were looking for — in a few lines of code. `ExecuteUpdate` commits at once when it runs alone; inside the transaction it waits for `CommitAsync`, and `SaveChanges` joins the transaction instead of starting its own. Two tests pin it: `One_transaction_covers_ExecuteUpdate_and_SaveChanges` and `A_failed_SaveChanges_rolls_back_the_ExecuteUpdate_too`, where a duplicate policy number makes the INSERT fail and the rivals stay `Quoted`. Dapper joins the same transaction through `db.Database.GetDbConnection()` and `tx.GetDbTransaction()` (`Dapper_joins_the_transaction_of_an_EF_Core_context`). **One production trap:** with a retrying execution strategy — SQL Server's `EnableRetryOnFailure`, for example — a user-started transaction throws until the whole unit runs inside `db.Database.CreateExecutionStrategy().ExecuteAsync(…)` so it can be replayed (Connection resiliency). Not run here: SQLite has no retrying strategy.

4.7 · Persistence-context habits — what carries over from Hibernate, what to drop

Carries over

- Unit of work: load, change, write once — `SaveChanges` is flush plus commit
- Identity map: one tracked instance per row per context (4.5)
- Optimistic locking with a version column and a conflict exception (4.4)
- Fetch plans chosen per use case — `Include` or a projection — not per entity
- Short transactions that never span user think-time

Drop or change

- Lazy loading as the default — navigations stay empty until included
- Flush-before-query — save first when a query must see the change
- `@Version` incrementing itself — stamp the token yourself, or map a database-generated one (`rowversion`, PostgreSQL `xmin`)
- Open Session in View — there is no `LazyInitializationException` to rescue; one context per request, never shared across threads
- Annotations as the mapping — configuration classes with the Fluent API

5 · Migrations — dotnet ef instead of Flyway and Liquibase

Flyway runs SQL you wrote; EF Core writes the migration for you from a model diff — the idea behind TypeORM's `migration:generate`, except that EF diffs against a committed snapshot. `dotnet ef migrations add` compares the current model with the committed `ModelSnapshot` and generates C# `Up` and `Down` methods. You review that C# in the pull request like any other code; SQL is produced from it for each provider. The three files per migration — migration, designer, updated snapshot — are all committed.

The tool is a pinned local tool, and a class library needs a design-time factory. `dotnet-ef` is restored from `dotnet-tools.json`, like a Gradle plugin version in the build. The tool builds your project and must create the context at design time; a library with no host provides an `IDesignTimeDbContextFactory` (EF Core tools reference). The `Microsoft.EntityFrameworkCore.Design` package is marked `PrivateAssets=all` so it never ships.

Two EF Core 9 changes make runtime migration safer and stricter. `Migrate()` now takes a database-wide lock, and it throws when the model has changes no migration covers (EF Core 9 breaking changes). Detect that earlier with `dotnet ef migrations has-pending-model-changes` in CI, or with a test like `Committed_migration_matches_the_model`, which

asserts `Database.HasPendingModelChanges()` is false.

For deployment, Microsoft recommends a migration bundle for automation and a SQL script when a person must review the SQL (Applying migrations). A bundle is a single-file executable built in CI and run as a one-shot job with schema-change credentials, while the application itself runs with least privilege — the same split you would make with a Flyway container before an ECS deploy.

5.1 · A generated migration — what you review in the pull request

C# — samples/L09.Data/Migrations/20260914121532_InitialCreate.cs · #region create-quotes

```
migrationBuilder.CreateTable(
    name: "Quotes",
    columns: table => new
    {
        Id = table.Column<int>(type: "INTEGER", nullable: false)
            .Annotation("Sqlite:Autoincrement", true),
        Reference = table.Column<string>(type: "TEXT", maxLength: 12, nullable: false),
        VehicleId = table.Column<int>(type: "INTEGER", nullable: false),
        Coverage = table.Column<string>(type: "TEXT", maxLength: 12, nullable: false),
        Status = table.Column<string>(type: "TEXT", maxLength: 10, nullable: false),
        ValidUntil = table.Column<DateOnly>(type: "TEXT", nullable: false),
        Version = table.Column<Guid>(type: "TEXT", nullable: false),
        Driver_ClaimsLast5Years = table.Column<int>(type: "INTEGER", nullable: false),
        Driver_DateOfBirth = table.Column<DateOnly>(type: "TEXT", nullable: false),
        Driver_LicenceYears = table.Column<int>(type: "INTEGER", nullable: false),
        Total_Amount = table.Column<decimal>(
            type: "TEXT", precision: 12, scale: 2, nullable: false),
        Total_Currency = table.Column<string>(type: "TEXT", maxLength: 3, nullable: false)
    },
    constraints: table =>
    {
        table.PrimaryKey("PK_Quotes", x => x.Id);
        table.ForeignKey(
            name: "FK_Quotes_Vehicles_VehicleId",
            column: x => x.VehicleId,
            principalTable: "Vehicles",
            principalColumn: "Id",
            onDelete: ReferentialAction.Restrict);
    });
```

Read it as you would read a Flyway script — it is the contract with production. The 29 lines above were generated, not typed: `maxLength`, `precision` and the `Restrict` foreign key all come from 2.2, and the complex-type columns are flattened with their prefixes. A rename in the model can appear as a drop and an add; fixing that by hand in the generated C# is normal.

5.2 · Four ways to apply migrations, next to the Flyway habit

Strategy	Flyway / Liquibase equivalent	SQL reviewable	SDK at deploy	Use for
`dotnet ef migrations script`	SQL handed to a DBA · Liquibase `update-sql`	● yes	● no	DBA-controlled or review-gated releases; add `--idempotent` where the provider supports it
`dotnet ef migrations bundle`	Flyway CLI in a one-shot container job	● no	● no	Automated pipelines — Microsoft's recommendation for automation
`dotnet ef database update`	`flyway migrate` from a laptop	● no	● yes	Local development and test databases only
`Database.MigrateAsync()` at start-up	`spring.flyway.enabled=true`	● no	● no	Simple deployments that accept start-up migration — the EF Core 9 lock makes several replicas safe, but the app then needs schema-change rights

Pill colour: green = what you want in production · amber = a cost to weigh · red = avoid in production.

5.3 · The CI gate, a SQL script that SQLite cannot make idempotent, and a bundle run twice

Terminal — dotnet ef in L09.Data captured · Build started/succeeded lines and long URLs left out

```
$ dotnet ef migrations has-pending-model-changes
No changes have been made to the model since the last migration.

$ dotnet ef migrations script --idempotent
Generating idempotent scripts for migrations is not currently supported for SQLite.

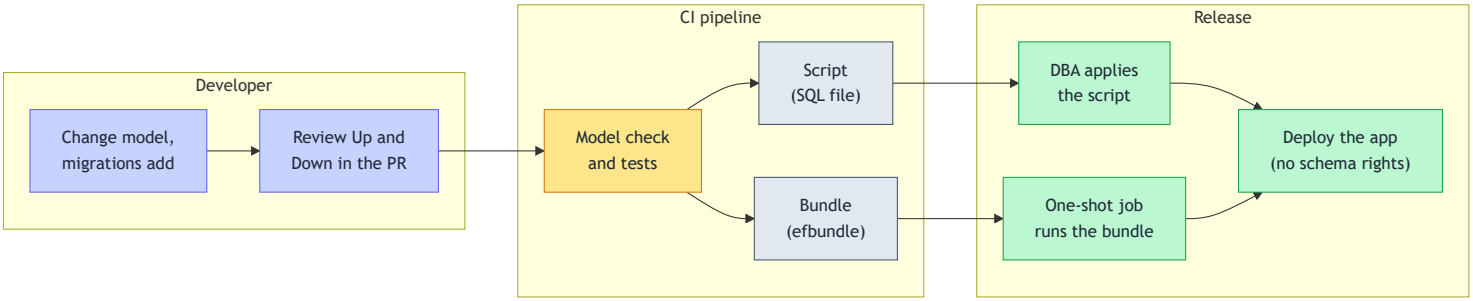
$ dotnet ef migrations bundle -o efbundle.exe
Building bundle...
Done. Migrations Bundle: ...\efbundle.exe

$ ./efbundle.exe --connection "Data Source=mq.db"
Acquiring an exclusive lock for migration application. ...
Applying migration '20260914121532_InitialCreate'.
Done.

$ ./efbundle.exe --connection "Data Source=mq.db"
Acquiring an exclusive lock for migration application. ...
No migrations were applied. The database is already up to date.
Done.
```

The bundle is idempotent even where the script is not. It records applied migrations in `__EFMigrationsHistory`, takes the EF Core 9 migration lock, and does nothing on the second run. SQLite cannot generate an idempotent script because it has no procedural SQL (SQLite provider limitations); the SQL Server provider can. The framework-dependent bundle built here was 33.0 MB • **Measured**.

5.4 · From a model change to production



indigo = developer · amber = automated check · slate = deployment artefact · green = runs against production · the app never holds schema-change credentials

6 · Beyond tracked entities — Dapper, bulk statements, providers and DynamoDB

Dapper is what you reach for when the SQL is the design. A reporting query with `COUNT(DISTINCT …)`, a `LEFT JOIN` and a `GROUP BY` — the kind of query you kept by the hundred behind BI dashboards — reads best as SQL. Dapper (Dapper) adds extension methods to any `DbConnection`, binds parameters from an anonymous object and maps columns to properties by name: Spring's `JdbcClient` without the template. It can share the connection, and the transaction, of an EF Core context (4.6).

`ExecuteUpdate` and `ExecuteDelete` are set-based writes inside EF. They translate a LINQ filter into one UPDATE or DELETE, load nothing, and ignore the change tracker — so tracked instances go stale, no concurrency token is checked, no transaction is started for you, and they work only on relational providers (ExecuteUpdate and ExecuteDelete). Expiring stale quotes is their textbook case.

Providers do not cross EF Core major versions, so the provider can pin your upgrade. For Aurora PostgreSQL, Npgsql shipped EF Core 10 support in November 2025 (Npgsql EF Core provider on NuGet). For Aurora MySQL, Oracle's `MySql.EntityFrameworkCore` has a 10.0 line (MySql.EntityFrameworkCore on NuGet), while the long-popular Pomelo provider's latest stable release still targets EF Core 9 (Pomelo MySQL provider on NuGet). Check both before choosing a .NET version for a MySQL estate.

DynamoDB has no EF Core provider. Use the AWS SDK for .NET directly: its `DynamoDBContext` maps attributed classes to items and supports optimistic locking, but it has no table-management API and nothing like LINQ-to-SQL (DynamoDB .NET object persistence model). Your DynamoDB access patterns and key design transfer unchanged; the ORM expectations do not.

6.1 · A reporting query — Spring JdbcClient vs Dapper

Java — for comparison Spring JdbcClient

```java
public List<CoverageRow> byCoverage(String status) {
    var sql = """
        SELECT q.coverage, COUNT(DISTINCT q.id) AS quotes,
               COUNT(l.id) AS lines
        FROM quotes q
        LEFT JOIN premium_lines l ON l.quote_id = q.id
        WHERE q.status = :status
        GROUP BY q.coverage
        ORDER BY q.coverage
        """;
    return jdbcClient.sql(sql)
        .param("status", status)
        .query(CoverageRow.class)
        .list();
}
```

C# samples/L09.Data/CoverageReport.cs · #region dapper

```csharp
public static async Task<List<CoverageRow>> ByCoverageAsync(
    DbConnection connection, string status)
{
    const string sql = """
        SELECT q.Coverage, COUNT(DISTINCT q.Id) AS Quotes,
               COUNT(l.Id) AS Lines
        FROM Quotes q
        LEFT JOIN PremiumLines l ON l.QuoteId = q.Id
        WHERE q.Status = @status
        GROUP BY q.Coverage
        ORDER BY q.Coverage
        """;
    var rows = await connection.QueryAsync<CoverageRow>(
        sql, new { status });
    return rows.AsList();
}
```

Almost a transliteration. `@status` is bound from the anonymous object `new { status }`; rows map to the `CoverageRow` record by column name, as `query(CoverageRow.class)` does. The console ran this and the equivalent EF Core `GroupBy` on the same data and printed identical counts, and a test asserts that they agree. Note the order both printed — `Class3` before `Class3Plus` — because `Coverage` is stored as a string.

6.2 · Expiring stale quotes — tracked SaveChanges vs one ExecuteUpdate

C# samples/L09.QueryConsole/Program.cs · #region expire-tracked

```csharp
// SELECT: 9 rows loaded and tracked
var stale = await db.Quotes
    .Where(q => q.Status == QuoteStatus.Quoted
             && q.ValidUntil < QuoteSeed.Today)
    .ToListAsync();
foreach (var q in stale)
    q.Status = QuoteStatus.Expired;  // in memory
// SaveChanges: one UPDATE statement per row
expired = await db.SaveChangesAsync();
```

C# samples/L09.Data/QuoteQueries.cs · #region bulk-expire

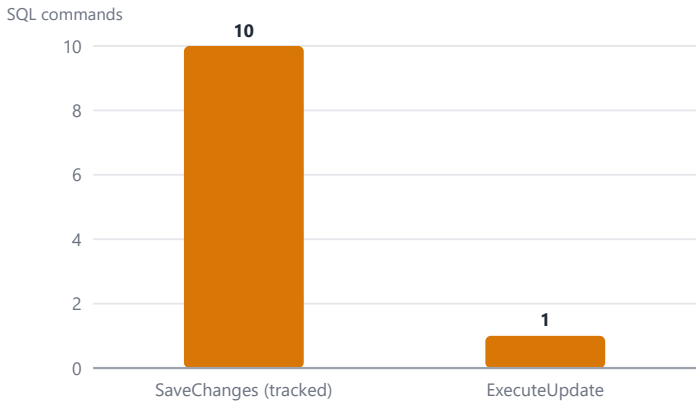
```csharp
// One UPDATE; nothing loaded or tracked. The
// token is NOT checked, so rotate it yourself.
public static Task<int> ExpireStaleAsync(
    this MotorQuoteDbContext db, DateOnly today)
{
    var stamp = Guid.NewGuid();
    return db.Quotes
        .Where(q => q.Status == QuoteStatus.Quoted
                 && q.ValidUntil < today)
        .ExecuteUpdateAsync(s => s
            .SetProperty(q => q.Status, QuoteStatus.Expired)
            .SetProperty(q => q.Version, stamp));
}
```

Same 9 rows changed; 10 commands against 1. The tracked version loads every stale quote, then sends one UPDATE per row on SQLite; SQL Server would batch those statements into fewer round trips (Efficient updating), but still loads and tracks the rows. The `ExecuteUpdate` version is a single `UPDATE … WHERE Status = 'Quoted' AND ValidUntil < @today`. Because it bypasses `SaveChanges`, it rotates `Version` itself so concurrent editors still get their conflict.

6.3 · Providers you will meet — and whether they support EF Core 10

Database	Package · maintainer	EF Core 10	Watch for
SQL Server · Azure SQL	`Microsoft.EntityFrameworkCore.SqlServer` · Microsoft	● yes	`rowversion` tokens; EF Core 10 adds the SQL Server 2025 `vector` and `json` types
PostgreSQL · Aurora PostgreSQL	`Npgsql.EntityFrameworkCore.PostgreSQL` · Npgsql team	● 10.0.3	`xmin` works as a concurrency token. Database providers still lists 8 and 9 — NuGet has the 10.0 line
MySQL · Aurora MySQL	`MySql.EntityFrameworkCore` · Oracle	● 10.0.9	Pomelo (`Pomelo.EntityFrameworkCore.MySql`): latest stable 9.0.0, EF Core 9 only
Oracle Database	`Oracle.EntityFrameworkCore` · Oracle	● yes	Listed for EF Core 8, 9 and 10
SQLite	`Microsoft.EntityFrameworkCore.Sqlite` · Microsoft	● yes	No schemas, sequences or database-generated tokens; `decimal` stored as TEXT
Amazon DynamoDB	none — AWS SDK for .NET `DynamoDBContext`	● no provider	Model access patterns and keys, not a relational schema
EF Core InMemory	`Microsoft.EntityFrameworkCore.InMemory` · Microsoft	● yes	Not a database: no transactions, no raw SQL, no `ExecuteUpdate`

6.4 · Commands to expire the stale quotes



9 of 20 seeded quotes expired · SQLite in-memory · measured

The tracked path scales with the number of rows; the bulk path does not. On SQLite that is one SELECT plus one UPDATE per row.
• **Measured**

6.5 · Which tool gives you what

	Typed	Tracking	Locking	Raw SQL	Schema
EF tracked	yes	yes	yes	manual	yes
EF no-tracking	yes	—	—	manual	yes
ExecuteUpdate	yes	—	manual	manual	yes
Dapper	manual	—	manual	yes	—
DynamoDBContext	manual	—	yes	—	—

yes = built in · manual = you write or wire it · — = not offered · a rubric, not a benchmark

No row is all green, so a service usually mixes them. Tracked entities for the quote workflow, projections for reads, `ExecuteUpdate` for housekeeping, Dapper for reports. Each column is a capability, not a goal — a dash under Tracking is what a read path wants. Typed = compiler-checked queries; Tracking = change tracking and `SaveChanges`; Locking = optimistic concurrency (DynamoDBContext's version attribute); Raw SQL = hand-written SQL is a supported way to use the tool; Schema = managed migrations.
• **Estimate**

7 · Testing data access — choose the fake that lies least

Microsoft's own order of preference surprises most JVM engineers: test against your production database engine first. The EF Core team recommends real-database tests (containers make that cheap), then stubbing a repository layer, then SQLite in-memory — and calls the EF Core InMemory provider *highly discouraged* as a test double (Choosing a testing strategy). Testcontainers for .NET (Testcontainers for .NET) is the Testcontainers you know — a package that starts a throwaway database container from the test process and hands you its connection string, which the 7.1 fixture would then open instead of `:memory:`. **No lesson in this track runs one:** every sample must build and pass offline with no Docker, so lesson 10 (Testing, TDD & 100% Coverage) lists it in its tooling table and declines it for the same reason. The tier is described here, never executed.

The InMemory provider is a dictionary, not a database. It ignores transactions, cannot run raw SQL, and does not support `ExecuteUpdate`. The 7.3 test proves the last point: code that is correct on every relational provider throws there.

SQLite in-memory is the H2 of this lesson — relational, fast, and still not your production engine. The connection *is* the database, so the fixture keeps one open for the test's lifetime and applies the committed migration. Differences remain: string comparison is case-sensitive where SQL Server is not, and `decimal` is TEXT (7.4). Use it for query shape and command counts; run the provider-specific behaviour against the real engine.

7.1 · The fixture — one open connection is one private database

C# — samples/L09.Data/SqliteDb.cs · #region sqlite-in-memory

```
// A private SQLite database in memory. The CONNECTION is the database:
// close it and the data is gone, so it stays open for this object's life.
public sealed class SqliteDb : IDisposable
{
    private readonly SqliteConnection _connection = new("Data Source=:memory:");

    public SqlCommandLog Log { get; } = new();
    public SqliteConnection Connection => _connection;

    public SqliteDb()
    {
        _connection.Open();
        using var db = NewContext();
        db.Database.Migrate();   // the committed migration, not EnsureCreated()
    }

    public MotorQuoteDbContext NewContext() =>
        new(new DbContextOptionsBuilder<MotorQuoteDbContext>()
            .UseSqlite(_connection)
            .AddInterceptors(Log)
            .Options);

    public void Dispose() => _connection.Dispose();
}
```

Close the connection and the database is gone, which is exactly the per-test isolation you want. `Migrate()` rather than `EnsureCreated()` means the schema under test is the one 5.1 generates; `AddInterceptors(Log)` wires in the command counter.

7.2 · Counting commands with a DbCommandInterceptor

C# — samples/L09.Data/SqlCommandLog.cs · #region interceptor

```
// Records every command EF Core sends, sync or async. Tests assert on
// Count (the N+1 is a number, not a feeling); the console prints Sql.
public sealed class SqlCommandLog : DbCommandInterceptor
{
    private readonly List<string> _sql = [];

    public int Count => _sql.Count;
    public IReadOnlyList<string> Sql => _sql;
    public void Reset() => _sql.Clear();

    public override ValueTask<InterceptionResult<DbDataReader>> ReaderExecutingAsync(
        DbCommand command, CommandEventData eventData,
        InterceptionResult<DbDataReader> result, CancellationToken cancellationToken = default)
    {
        _sql.Add(command.CommandText);
        return ValueTask.FromResult(result);
    }

    // ...plus ReaderExecuting, NonQueryExecuting(Async) and ScalarExecuting(Async),
    // the same shape: sync and async paths are separate overrides.
```

An interceptor sees every `DbCommand` EF Core executes — the same hook you would put a Hibernate `StatementInspector` or a datasource proxy on. Override both the sync and async variants, or a test that awaits will count nothing.

7.3 · The InMemory provider trap, pinned by a test

C# — samples/L09.Data.Tests/ProviderTrapTests.cs · #region in-memory-trap

```
[Fact]
public async Task The_InMemory_provider_cannot_run_ExecuteUpdate()
{
    var options = new DbContextOptionsBuilder<MotorQuoteDbContext>()
        .UseInMemoryDatabase("trap")
        .Options;
    await using var db = new MotorQuoteDbContext(options);

    // Compiles, and runs on every relational provider (SQLite here):
    // on InMemory it throws.
    var ex = await Assert.ThrowsAsync<InvalidOperationException>(
        () => db.ExpireStaleAsync(QuoteSeed.Today));
    Assert.Contains("ExecuteUpdate", ex.Message);
}
```

Green on SQLite in this suite, an exception on InMemory • Measured. The message names `ExecuteUpdate` because the provider cannot translate it — it is supported on relational providers only (ExecuteUpdate and ExecuteDelete). A suite built on InMemory would have had to skip the one method that matters most for bulk housekeeping.

7.4 · SQLite stores decimal as TEXT — raw SQL sorts it as text

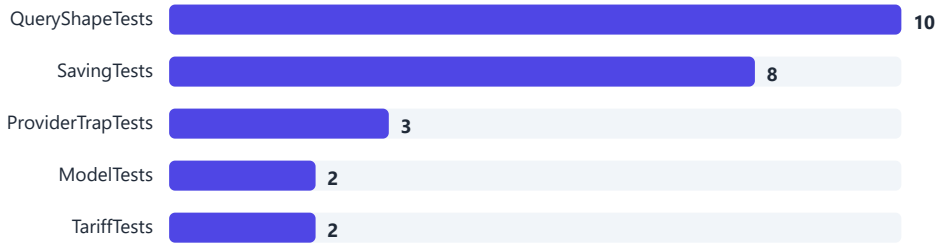
C# — samples/L09.QueryConsole/Program.cs · #region raw-text-order

```
var raw = await sqlite.Connection.QueryAsync<string>("""
    SELECT Reference || ' ' || Total_Amount FROM Quotes
    ORDER BY Total_Amount DESC LIMIT 3
    """);   // the column holds TEXT: this sorts strings

var smallest = await sqlite.Connection.QueryAsync<string>("""
    SELECT Reference || ' ' || Total_Amount FROM Quotes
    ORDER BY CAST(Total_Amount AS REAL) LIMIT 3
    """);   // the workaround: sort as a number
```

EF Core hides the difference; raw SQL does not. EF Core translated `OrderByDescending(q => q.Total.Amount)` to `ORDER BY … COLLATE EF_DECIMAL DESC` and returned the three largest premiums (Q-0005, Q-0013, Q-0010). The raw SQL on the same TEXT column returned Q-0017 9159.31, Q-0006 6703.51, Q-0018 5736.66 — `'9'` sorts after `'2'` as text — and `CAST(Total_Amount AS REAL)` restored numeric order, smallest first: Q-0007 0.0, Q-0014 0.0, Q-0012 1744.64 (the two zeros are the declined quotes — 3 or more claims are never priced) • Measured. The `CAST` is a workaround for this fixture only: on SQL Server and PostgreSQL `decimal` is a native numeric type and the trap does not exist. SQLite provider limitations still says such comparisons run on the client; the SQL captured from EF Core 10.0.12 shows them translated.

7.5 · Test cases by class



[Fact] methods + [InlineData] rows in L09.Data.Tests · counted at build · measured

25 cases, and most assert a number of SQL commands or a translation — not just a result. That is what makes a data-access test catch an N+1 or an IEnumerable leak. • Measured

7.6 · What the application may still do — for each kind of test double

Test double	Raw SQL	Transactions	Provider functions	Exact query behaviour	LINQ anywhere	Verdict
InMemory provider	• no	• no · ignored	• no	• depends	• yes	Highly discouraged by the EF Core team
SQLite in-memory	• depends	• yes	• no	• depends	• yes	Query shape, command counts, migrations — this lesson's fixture
Stubbed repository	• yes	• yes	• yes	• yes	• no *	Unit tests above the data layer — the LINQ inside the repository is not tested
Real engine (container)	• yes	• yes	• yes	• yes	• yes	Microsoft's first recommendation — needs Docker in CI

How to read it. A "yes" means the application may use that feature and the test still works, not that the double verifies it: a stubbed repository replaces the LINQ, so nothing translates it. *** The price of the repository:** every testable query must sit behind an `IEnumerable`-returning method, and tests against the real database are still needed for those queries. Rows and values follow Microsoft's overall comparison (Choosing a testing strategy).

8 · Visual Basic alongside — the runtime is shared, the tooling is not

EF Core's runtime does not care which language built the expression tree. VB's query syntax compiles to the same `IQueryable` calls as C#, so a VB report can query the C# `MotorQuoteDbContext` directly — and VB's `Group By … Into` becomes `GROUP BY` in SQL. VB even compiles string `=` to a call to `Operators.CompareString`, and EF Core still translates it to a plain SQL `=`, as the captured output shows.

The design-time tools generate C# only. A VB project can declare a `DbContext` and run it, but `dotnet ef migrations add` and `dotnet ef dbcontext scaffold` both refuse a VB project (8.3). The community package that added VB generators stops at EF Core 8 (EntityFrameworkCore.VisualBasic on NuGet). The tool's own advice is the architecture to use: keep the model and migrations in a C# library and reference it from VB — the migration path lesson 06 (VB.NET for Legacy Estates) recommends anyway.

Estates on .NET Framework usually run EF6, not EF Core. EF6 is stable and supported but no longer actively developed, and EF Core is not a drop-in replacement: EDMX models, Entity SQL and the designer have no EF Core equivalent (Compare EF6 and EF Core). Keep EF6 where the data code is stable; port the parts that must evolve (lesson 12 (Cloud-Native .NET on AWS & Modernization)).

8.1 · One grouping query — C# method syntax vs VB query syntax

C# `samples/L09.QueryConsole/Program.cs · #region cs-group`

```
var byCoverage = await db.Quotes
    .Where(q => q.Status == QuoteStatus.Quoted)
    .GroupBy(q => q.Coverage)
    .Select(g => new { Coverage = g.Key, Quotes = g.Count() })
    .OrderBy(x => x.Coverage)
    .ToListAsync();
```

Visual Basic `samples/L09.VbReport/Program.vb · #region vb-query`

```
' VB query syntax. Group By ... Into becomes SQL GROUP BY;
' nothing runs until the For Each enumerates the query.
Dim byCoverage = From q In db.Quotes
                 Where q.Status = QuoteStatus.Quoted
                 Group By q.Coverage Into Quotes = Count()
                 Order By Coverage

For Each row In byCoverage
    Console.WriteLine($"  {row.Coverage,-11}{row.Quotes,3}")
Next
```

Same SQL from both. VB's `Group By q.Coverage Into Quotes = Count()` names the aggregate inline, where C# needs a `Select` into an anonymous type. The VB query runs when `For Each` enumerates it — deferred execution is identical. Both printed the same four rows.

8.2 · What EF Core receives from VB — and the SQL it sends

Visual Basic samples/L09.VbReport/Program.vb · #region vb-strings

```
' VB compiles = on two strings to Operators.CompareString.
Dim toyotas = From v In db.Vehicles
              Where v.Make = "Toyota"
              Select v.Model
```

SQL — ToQueryString() from VB captured from L09.VbReport

```
SELECT "q"."Coverage", COUNT(*) AS "Quotes"
FROM "Quotes" AS "q"
WHERE "q"."Status" = 'Quoted'
GROUP BY "q"."Coverage"
ORDER BY "q"."Coverage"
SELECT "v"."Model"
FROM "Vehicles" AS "v"
WHERE "v"."Make" = 'Toyota'
```

The VB-specific call disappears in translation. The first statement is the 8.1 query; the second is the `Make = "Toyota"` filter, sent as `WHERE "v"."Make" = 'Toyota'` even though VB compiled the comparison to `CompareString`. Case rules are the database's: case-sensitive on SQLite, case-insensitive under SQL Server's default collation (Choosing a testing strategy).

8.3 · The EF Core tools refuse a VB project

Terminal — dotnet ef in a VB project captured · long lines wrapped · build lines left out

```
$ cd L09.VbModel
$ dotnet ef migrations add Initial
The project language 'VB' isn't supported by the built-in
IMigrationsCodeGenerator service. You can try looking for an additional
NuGet package which supports this language; moving your DbContext type
to a C# class library referenced by this project; or manually implementing
and registering the design-time service for the programming language.
# exit code 1

$ dotnet ef dbcontext scaffold "Data Source=mq.db" Microsoft.EntityFrameworkCore.Sqlite
The project language 'VB' isn't supported by the built-in
IModelCodeGenerator service. ...
# exit code 1
```

The context in `L09.VbModel` compiles and would run — only the code generators are missing. Both commands build the project first, then fail when they look for a VB code generator. For a VB estate this settles where new schema work lives: a C# data library.

9 · Hands-on — run the lesson-09 samples

Five projects, one data library. `L09.Data` holds the context, configuration, committed migration, queries, the Dapper report and the SQLite fixture. The console prints the SQL behind each trap in this lesson; the tests turn the same traps into assertions; the two VB projects show what VB can and cannot do with EF Core.

Read the console output against the sections. Block 2 is chart 3.4, block 3 is 3.3, block 5 is 6.2, block 6 is 6.1 — and block 7 is 7.4, the decimal-as-TEXT trap.

9.1 · Commands

Terminal

```
# from the repository root
cd lesson-09-data-access-efcore/samples
dotnet tool restore                      # dotnet-ef 10.0.12 from dotnet-tools.json

dotnet run --project L09.QueryConsole    # SQL for every trap in this lesson
dotnet run --project L09.VbReport        # the C# context queried from VB
dotnet test L09.Data.Tests               # 25 cases on SQLite in-memory

# migrations, from the data library
cd L09.Data
dotnet ef migrations has-pending-model-changes
dotnet ef migrations script -o ../../../.build/l09/migrate.sql
dotnet ef migrations bundle -o ../../../.build/l09/efbundle.exe

# or, from the repository root: build, test and run every lesson-09 sample
python 0-script/verify_samples.py --only 9
```

`dotnet tool restore` is the step people miss. Without it `dotnet ef` is not found, because the tool is pinned locally rather than installed globally. The script and bundle outputs go to the git-ignored `.build` folder.

9.2 · What you should see

Output — L09.QueryConsole (excerpt) captured on the build machine

```
EF Core 10.0.12 on SQLite in-memory; illustrative rates

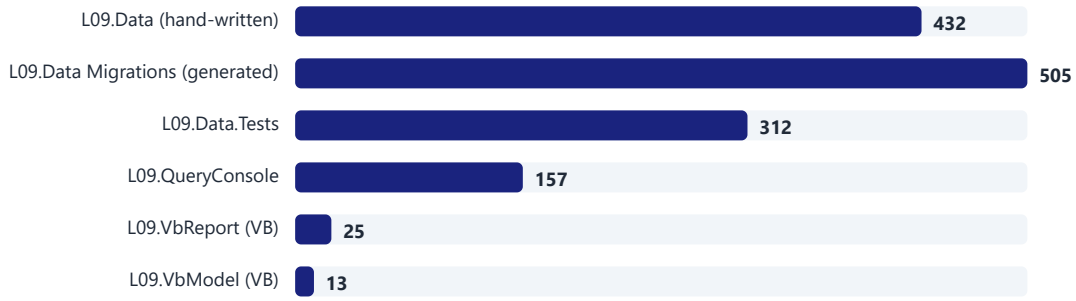
== 2. Commands to load N quotes with their premium lines
  quotes   N+1   Include   AsSplitQuery
       1     2         1              2
       5     6         1              2
      10    11         1              2
      20    21         1              2

== 5. Expire stale quotes: tracked SaveChanges vs ExecuteUpdate
  tracked + SaveChanges  rows  9  commands 10
  ExecuteUpdate          rows  9  commands  1

== 7. decimal is stored as TEXT on SQLite
  EF Core  top 3 by total  Q-0005, Q-0013, Q-0010
  EF Core  totals over 20,000: 2
  raw SQL  top 3 by total  Q-0017 9159.31, Q-0006 6703.51, Q-0018 5736.66
  raw SQL  3 smallest (CAST AS REAL)  Q-0007 0.0, Q-0014 0.0, Q-0012 1744.64
```

> **Every number here is deterministic.** The seed data is fixed and SQLite runs in process, so your counts must match; blocks 1, 3, 4 and 6 appear in 3.2, 3.3, 3.8 and 6.1, and block 7 is the trap explained in 7.4 — EF Core's top three are the three largest premiums, raw SQL is not.

9.3 · Lines of code in the lesson-09 samples



non-blank, non-comment lines · measured from the sample files when this PDF was built

> **The first migration, its designer file and the model snapshot are 505 lines — 1.2× the hand-written data library (432 lines).** They are generated, but you still review and commit them; a later migration adds only its own diff. • Measured — a size signal, not a quality score.

⚠ Traps

1. **"The collection will lazy-load."** With lazy loading off — the default — a missing `Include` leaves `quote.Lines` empty — no proxy, no exception, just wrong totals (test `Missing_Include_leaves_Lines_empty`).
2. **"A repository can return `IEnumerable`."** Only after materialising. Any LINQ applied to an `IEnumerable` view of a `DbSet` reads and tracks the whole table (20 entities for one count in 3.3).
3. **"The query will see what I just changed."** EF Core does not flush before a query; save first (4.5).
4. **"`IsConcurrencyToken` is `@Version`."** It only adds the `WHERE` check. Something must change the value on every write — a database-generated token (`rowversion` on SQL Server, `xmin` on PostgreSQL) or your own code, as on SQLite — and `ExecuteUpdate` skips both the check and your override.
5. **"Tests on the InMemory provider prove the data layer."** It has no transactions, raw SQL or `ExecuteUpdate`; use SQLite in-memory for query shape and a real engine for the rest.

✓ Checkpoint — you are ready for lesson 10 when

1. `python 0-script/verify_samples.py --only 9` reports 5/5 passed on your machine.
2. Given a LINQ query, you can predict its SQL and confirm it with `ToQueryString()`.
3. You can explain every bar in chart 3.4 and write the test that keeps an N+1 from coming back.
4. You can say which of script, bundle, `database update` and `MigrateAsync` you would use in a regulated production pipeline, and why.
5. You can place a VB estate's data access: EF6 kept or ported, and new models in a C# library.

🔍 Check yourself

1. Why is a `DbContext` registered per request scope, what happens when two tasks use one instance at the same time, and what do you inject for parallel or background work?
2. What SQL does `Count` send when `db.Quotes` is typed as `IEnumerable`, and how many entities end up tracked?
3. Why does 6.2's `ExecuteUpdate` set `Version` itself, when the `SaveChangesAsync` override in 4.3 stamps it on tracked writes?
4. Which EF Core release made struct complex types possible, and what does Microsoft now advise for code that uses owned entity types?
5. What two things does `Migrate()` do differently since EF Core 9?
6. Name two things the InMemory provider cannot do that SQLite in-memory can (7.3, 7.6).
7. A nightly job expires stale quotes and a dashboard needs a `GROUP BY` report — which tool for each, and what does each give up (6.5)?
8. Two writes must succeed or fail together, and one of them is an `ExecuteUpdate` — what wraps them (4.6)?

Lesson 09 in one line: EF Core is a short-lived unit of work that turns LINQ into SQL — count the commands, project your reads, never flush-by-query, stamp your concurrency tokens, generate and review migrations, and reach for `ExecuteUpdate`, Dapper or the AWS SDK when tracked entities are the wrong tool.
Next: lesson 10 (Testing, TDD & 100% Coverage) — xUnit, coverage gates and the integration tests that sit on top of this data layer.